Predicting Flight Arrival Delays from New York City Airports

HarvardX PH125.9x Data Science: Capstone – Choose Your Own Project

Cristiano Camilo de Araujo Lelis

2026-10-23

Contents

1 Introduction **1**

2 Methods and Analysis **2**

2.1 Data cleaning 2

2.2 Train and final test split 3

2.3 Data exploration 4

2.4 Feature engineering and rare categories 11

2.5 Validation strategy 11

2.6 Models 12

3 Results **15**

3.1 Model comparison on the validation set 15

3.2 Variable importance 16

3.3 Final model and evaluation on the final test set 16

4 Conclusion **17**

5 References **18**

1 Introduction

Arrival delays are an operational and financial problem for commercial aviation. Delays impose substantial costs on passengers, on airlines (crew, fuel and maintenance) and on the economy as a whole, as shown by a comprehensive study sponsored by the U.S. Federal Aviation Administration (Ball et al., 2010). In regions with very dense air traffic, such as the New York City metropolitan area, the delay of a single flight can spread to other flights through the network. Although airport congestion and severe weather remain major causes of delays, a model that estimates the risk of a late arrival before departure could help airlines plan their ground operations in advance and help passengers reduce risks on tight connections. This project develops such a model, using only scheduling information that is known before departure.

The data come from the Kaggle dataset "Flights", published under a CC0 (public domain) license, whose original source is the U.S. Bureau of Transportation Statistics (BTS). It contains all 336,776 flights that departed from the three New York City airports (EWR, JFK and LGA) in 2013, with 21 variables, including scheduled and actual departure and arrival times, delays, carrier, flight number, aircraft tail number, origin, destination, air time and distance. The same flight records are also distributed in the `nycflights13` R package (Wickham), which is widely used in R tutorials, mainly to teach data wrangling and visualization rather than to predict delays, and is not used in the course textbook; the Kaggle version adds an index column (`id`) and the full carrier name (`name`). The table below describes the variables used in the analysis, after the data preparation described in the Methods section.

Table 1: Variables used in the analysis

Variable	Role	Type	Description
delayed	Target	Factor (no, yes)	Arrival delay above 15 minutes
month	Predictor	Factor (1-12)	Month of the flight
weekday	Predictor	Factor (Mon-Sun)	Day of the week (from the date)
sched_hour	Predictor	Numeric (5-23)	Scheduled departure hour
carrier	Predictor	Factor (16 levels)	Airline code
origin	Predictor	Factor (3 levels)	NYC departure airport
dest	Predictor	Factor (92 levels)	Destination (rare ones as "Other")
distance	Predictor	Numeric (miles)	Flight distance

The main goal of this project is to predict whether a flight will arrive more than 15 minutes late at its destination, using only information that is already known before departure. This restriction is essential for the model to be useful in practice and to avoid data leakage: variables that are common in retrospective analyses, such as the actual departure time or the departure delay, were discarded because they are unknown at the moment the prediction must be made. Since the data are imbalanced, with about three quarters of the flights arriving on time, performance is not measured by overall accuracy but by balanced accuracy, a metric that gives the same weight to correctly identifying on-time flights and delayed flights. As the main result, the final random forest model correctly identified about 64% of the delayed flights in the independent final test set.

To reach this result, the project followed six consecutive steps. First, the data were cleaned by removing cancelled and diverted flights, and the binary target variable was defined following the BTS standard. Second, the clean data were split, with stratification, into a development set (90%) and a final test set (10%). Third, the variables were explored in detail to identify patterns and to investigate possible confounding factors through stratification. Fourth, feature engineering replaced the day of the month with the day of the week, extracted the scheduled departure hour and grouped rare destinations. Fifth, four models were trained and compared (a majority-class baseline, a logistic regression, a CART decision tree and a random forest), using 5-fold cross-validation to tune the hyperparameters and to choose the best classification threshold for each model. Finally, the best model was retrained on the complete development set and evaluated a single time on the final test set.

2 Methods and Analysis

2.1 Data cleaning

Table 2: Variables with missing values in the original data

Variable	Missing values
dep_time	8,255
dep_delay	8,255
arr_time	8,713
arr_delay	9,430
tailnum	2,512
air_time	9,430

The missing values are concentrated in the variables that describe what actually happened during the flight. The 8,255 flights without an actual departure time are cancelled flights: since the aircraft never took off, no real times were recorded. The arrival delay is missing for 9,430 flights, that is, about 1,175 more flights than the cancelled ones. These flights departed but have no recorded arrival delay, most likely because they were diverted to other airports. Since the goal is to predict delays of flights that completed their original route, the cancelled and diverted flights were removed. After cleaning, 327,346 flights remain.

The target variable, `delayed`, is a binary factor that takes the value "yes" when a flight arrived more than 15 minutes late, following the standard used by the U.S. Bureau of Transportation Statistics, and "no" otherwise.

To build a realistic model, only variables known before departure were kept as predictors. The variables `dep_time`, `dep_delay`, `arr_time` and `air_time` were deliberately discarded: since they are only known after departure or arrival, including them would cause data leakage and hide the model's real ability to predict the risk of delay before the operation begins. Other variables were removed because they are identifiers or redundant: `id`, `flight` and `tailnum` are identifiers or variables with thousands of distinct values; `name` repeats the information in `carrier`; `time_hour` combines the date and hour variables; `year` is constant (all flights are from 2013); and `hour` and `minute` are components of `sched_dep_time`. The scheduled arrival time (`sched_arr_time`), although known before the flight, was also left out, because it is largely determined by `sched_dep_time` and `distance`. The seven predictors kept at this stage were `month`, `day`, `sched_dep_time`, `carrier`, `origin`, `dest` and `distance`.

The proportion of delayed flights is 23.7%. A model that always predicts "not delayed" would therefore reach an accuracy of about 76.3% without learning anything, which is why balanced accuracy, sensitivity and specificity are used to evaluate the models (Section 27.4 of the course textbook).

2.2 Train and final test split

The clean data were split into a development set (`dev_set`, 90%, 294,611 flights) and a final test set (`final_test`, 10%, 32,735 flights). The split was stratified by the target variable, so both sets keep practically the same proportion of delayed flights (about 23.7%). The months were also balanced between the two sets: the largest difference in the share of flights of any month was 0.4 percentage points.

The 90/10 proportion was chosen for two reasons. First, with about 32,700 flights in the final test set, the 95% margin of error of an accuracy around 76% is approximately $\pm$ 0.5 percentage points, which is precise enough to evaluate the final model; reserving 20% would reduce this margin only slightly while removing data from training. Second, delays are a minority event, and keeping 90% of the data for development maximizes the examples available for the models to learn from. Because of computing time, the heavier models were trained on a sample of the development set, as described in the validation strategy below. The `final_test` set was kept isolated during the whole development process and used only once, at the end, to evaluate the final model.

2.3 Data exploration

All exploration was done on the development set only.

2.3.1 Month

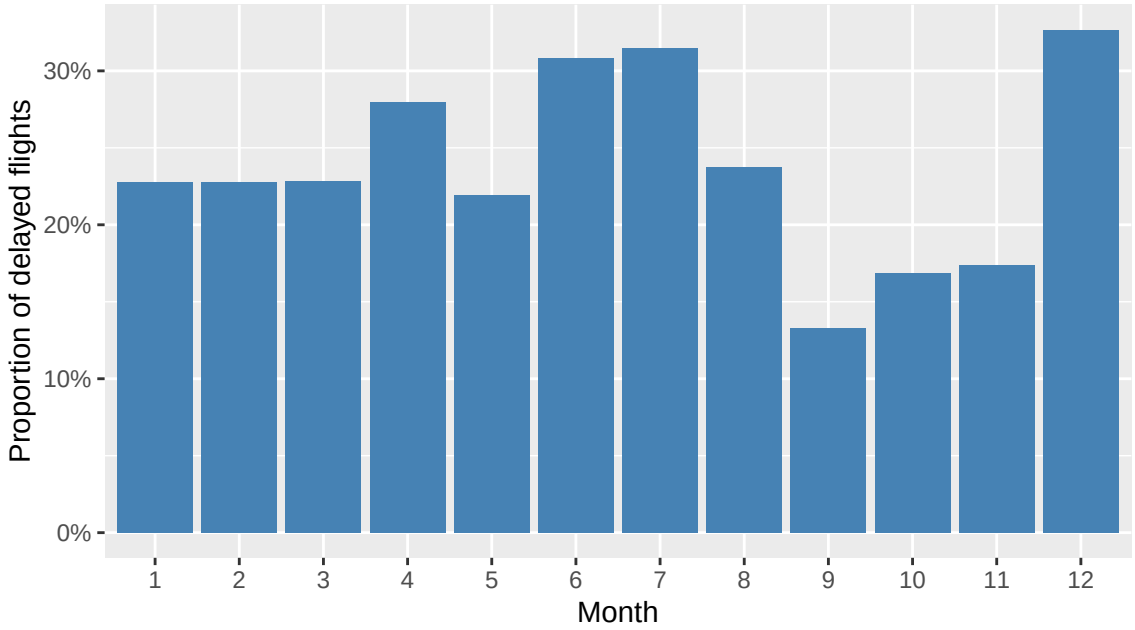


Figure 1: Delay rate by month (development set)

December has the highest delay rate (about 33%), followed by a high plateau in June and July (above 30%) and an intermediate peak in April (about 28%). September is the most punctual month (about 13%), followed by low and stable rates in October and November (about 17%). The dataset has no information on weather, airport capacity or seat occupancy, so the causes cannot be verified, but these patterns are consistent with known dynamics of the U.S. airline industry: December combines the heavy holiday traffic with the start of winter in the northeastern United States, when snowstorms require slow safety procedures such as aircraft de-icing; June and July combine the peak of leisure travel during the summer holidays with frequent afternoon thunderstorms on the East Coast; and from September to November, leisure demand falls after the return to school, in a season of more stable weather.

Since the delay rate does not follow an increasing or decreasing trend over the year, `month` was used as a categorical variable (factor) in the models, and not as a number, which would force models such as the logistic regression to fit an invalid linear trend across months.

2.3.2 Scheduled departure hour

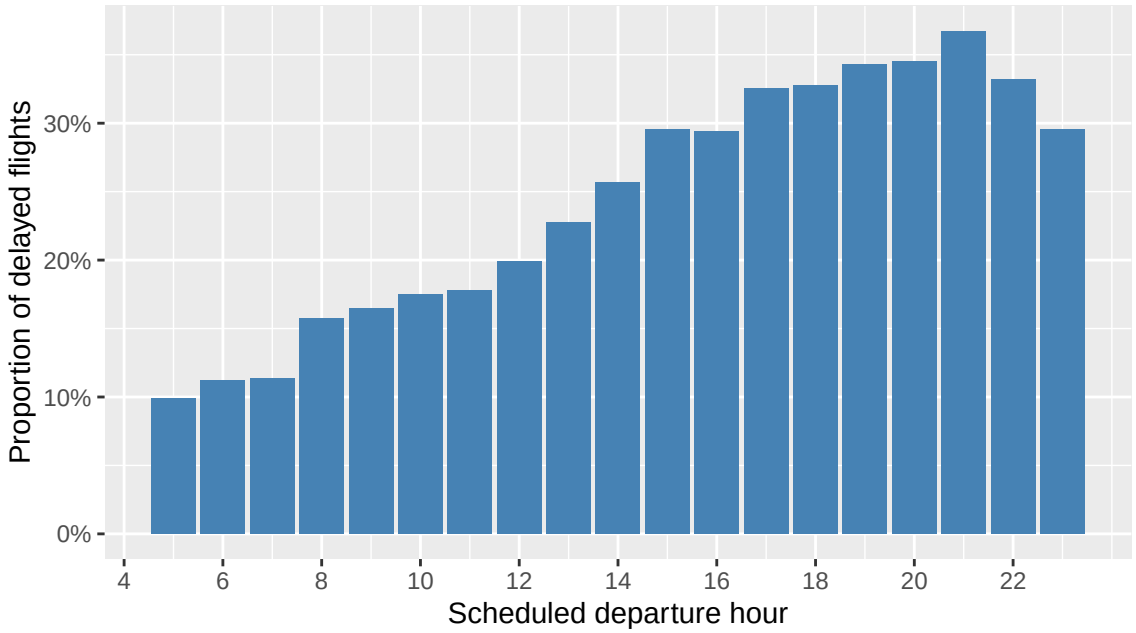


Figure 2: Delay rate by scheduled departure hour (development set)

The scheduled departure time is stored in a compact format (for example, 1530 for 3:30 p.m.), so the scheduled hour was extracted with `floor(sched_dep_time / 100)`. The delay rate grows steadily during the day: flights scheduled between 5 and 7 a.m. have the lowest rates (about 10% to 11%), and the rate rises in an approximately linear way until it peaks between 7 and 9 p.m. (34% to 37%). A slight decrease appears from 10 p.m. onward, but it should be interpreted with caution: the 5 a.m., 10 p.m. and 11 p.m. slots have only 1,732, 2,320 and 948 flights, against roughly 9,000 to 24,000 in the other hours, so their rates are less reliable.

The dataset does not track the previous flights of each aircraft, but this pattern is consistent with two operational realities. Early-morning flights usually use aircraft that spent the night at the airport, starting the day without delays inherited from previous flights. Later in the day, small delays accumulate across the several flights operated by the same aircraft (the so-called cascade effect), and afternoon summer thunderstorms may make this worse. Unlike the month, the delay rate shows a clear trend over the day, so the scheduled hour (`sched_hour`) was used as a numeric variable.

2.3.3 Origin airport

Table 3: Delay rate by origin airport

Origin	Flights	Delay rate
EWR	105,396	25.6%
JFK	98,240	23.0%
LGA	90,975	22.4%

The difference between the airport with the lowest delay rate (LGA, 22.4%) and the one with the highest (EWR, 25.6%) is only about 3 percentage points, much smaller than the variation across months or hours.

With about 100,000 flights per airport and a delay rate close to 25%, the standard error of each rate is only about 0.14 percentage points, so a 3-point difference is very unlikely to be due to chance: the difference is statistically real, but its practical size is modest. EWR is also the airport with the most departures in the development set. A possible explanation for its higher rate is congestion, but another is that each airport is the base of different carriers, which is investigated below. The variable `origin` has only three levels and was used as a factor.

2.3.4 Carrier

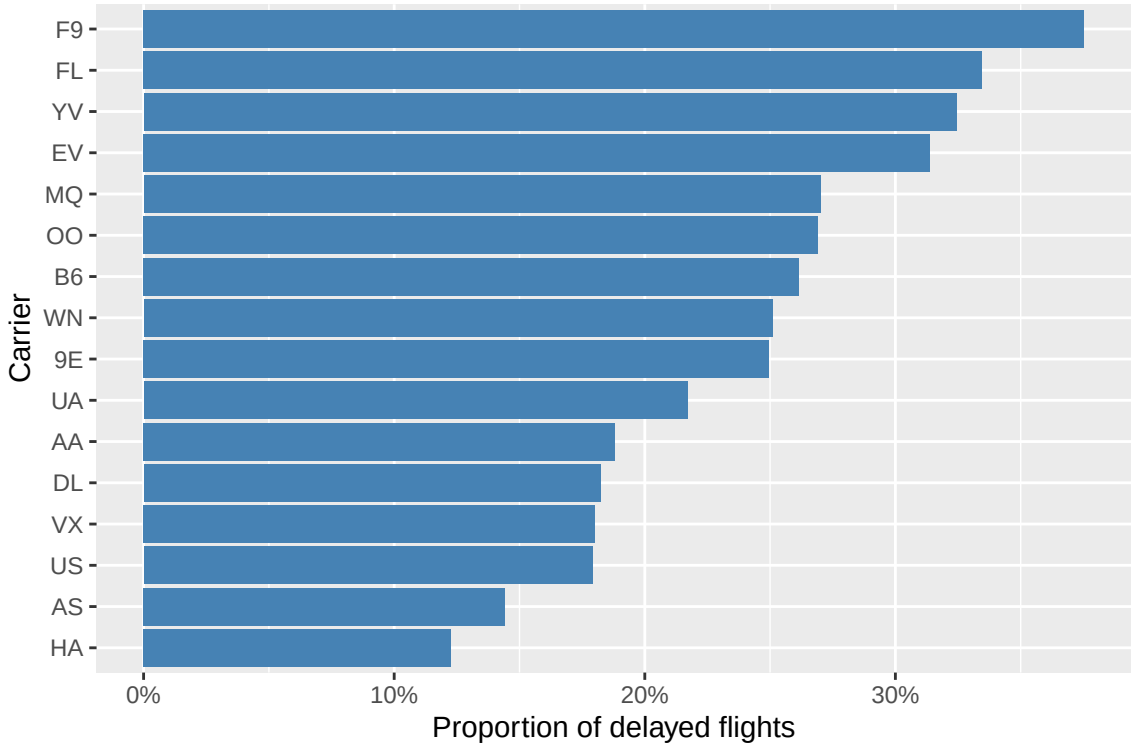


Figure 3: Delay rate by carrier (development set)

Table 4: Delay rate by carrier (development set)

Code	Carrier	Flights	Delay rate
F9	Frontier Airlines Inc.	618	37.5%
FL	AirTran Airways Corporation	2,873	33.4%
YV	Mesa Airlines Inc.	484	32.4%
EV	ExpressJet Airlines Inc.	46,034	31.4%
MQ	Envoy Air	22,568	27.0%
OO	SkyWest Airlines Inc.	26	26.9%
B6	JetBlue Airways	48,600	26.2%
WN	Southwest Airlines Co.	10,836	25.1%
9E	Endeavor Air Inc.	15,604	24.9%
UA	United Air Lines Inc.	51,959	21.7%
AA	American Airlines Inc.	28,745	18.8%
DL	Delta Air Lines Inc.	42,951	18.2%

Code	Carrier	Flights	Delay rate
VX	Virgin America	4,598	18.0%
US	US Airways Inc.	17,766	17.9%
AS	Alaska Airlines Inc.	631	14.4%
HA	Hawaiian Airlines Inc.	318	12.3%

The carriers show large differences. Frontier Airlines (F9) has the highest delay rate (37.5%), followed by AirTran Airways (FL), Mesa Airlines (YV) and ExpressJet Airlines (EV), while Hawaiian Airlines (HA) and Alaska Airlines (AS) have the lowest rates (12.3% and 14.4%). The raw range of 25.2 percentage points is, however, inflated by small carriers, which occupy both ends of the ranking. Considering only the carriers with more than 10,000 flights, the range falls to 13.5 points, from EV (31.4%) to US Airways (17.9%). The carrier therefore appears to be a relevant predictor, stronger than the origin airport, although weaker than the month or the scheduled hour.

The reliability of the small carriers varies. SkyWest Airlines (OO) has only 26 flights, which gives a 95% margin of error of about $\pm$ 17 percentage points, so its rate is not reliable. Hawaiian Airlines (318 flights) and Frontier Airlines (618 flights) have margins of about $\pm$ 4 points: their rates are imprecise, but still informative. For large carriers such as EV and JetBlue (B6), with more than 46,000 flights each, the high delay rates are a consistent pattern supported by a large volume of data. The variable `carrier` was used as a factor; the case of OO is discussed in the feature engineering section.

2.3.5 Confounding between airport and carrier

Table 5: Delay rate of each carrier by origin airport (groups with at least 1,000 flights)

Carrier	EWR	JFK	LGA
9E	15.7%	26.6%	20.1%
AA	18.7%	20.2%	17.5%
B6	24.0%	26.0%	29.6%
DL	20.7%	16.7%	19.1%
EV	32.4%	27.6%	26.6%
FL	—	—	33.4%
MQ	30.6%	27.1%	26.5%
UA	21.8%	21.2%	21.2%
US	16.5%	17.3%	18.6%
VX	17.3%	18.3%	—
WN	25.7%	—	24.5%

Each New York airport is dominated by different carriers: EWR by United (UA, 38.9% of its departures) and ExpressJet (EV, 35.5%); JFK by JetBlue (B6, 38.2%); and LGA is more evenly divided among Delta, Envoy Air, American and US Airways. EV, one of the carriers with the highest delay rates, operates more than 81% of its flights from EWR. This composition is consistent with confounding: part of EWR's higher delay rate could reflect the carriers that operate there, rather than the airport itself (see Sections 18.2 and 19.4 of the course textbook).

To investigate this, the delay rate of the same carrier was compared across airports (stratification, considering only groups with at least 1,000 flights). The evidence is mixed. There is no systematic airport effect: five carriers are more often late at EWR (DL, EV, MQ, UA and WN), while four are less often late there (9E, B6, US and VX). The variation within each carrier is small for UA, VX, WN and US, but large for 9E (about 11 points) and B6 (about 6 points). The stratification therefore suggests that part of the difference

between airports comes from the mix of carriers, since EV has high delay rates at all three airports, but also that the airport effect is not uniform: it depends on the carrier. This interaction between carrier and airport is an important argument for tree-based models, which can represent interactions naturally, while a simple logistic regression only adds separate effects for carrier and airport.

2.3.6 Destination

Table 6: Destinations with the highest and lowest delay rates (at least 1,000 flights)

Group	Destination	Flights	Delay rate
Highest	RIC	2,123	35.5%
Highest	MCI	1,725	32.0%
Highest	GSO	1,335	30.9%
Highest	JAX	2,393	29.9%
Highest	CVG	3,363	29.6%
Lowest	MIA	10,423	18.2%
Lowest	BOS	13,429	18.3%
Lowest	SLC	2,220	18.5%
Lowest	LAS	5,370	19.0%
Lowest	SRQ	1,074	19.5%

There are 104 distinct destinations. Among the destinations with at least 1,000 flights, the delay rate ranges from 18.2% (Miami, MIA) to 35.5% (Richmond, RIC), a difference of 17.3 percentage points, which suggests that the destination is relevantly associated with delays. Several destinations at the top of the ranking, such as RIC, GSO, CVG, JAX and MCI, are medium-sized cities typically served by regional flights, which is the profile of carriers such as ExpressJet (EV), whose delay rates are high at all three New York airports. Part of their high rates may therefore be associated with the carriers that operate these routes, a hypothesis that was not verified directly.

This high cardinality requires care in the modeling. In a logistic regression, rare destinations produce unstable coefficients, and a destination present in the test set but absent from the training data would make prediction impossible. In addition, the classic `randomForest` package for R does not accept categorical variables with more than 53 levels, which led to the use of the `ranger` package. Finally, 13 destinations have fewer than 100 flights in the development set: the smaller the number of flights, the larger the influence of each individual flight on the estimated rate, so these rates tend to reflect random variation more than an operational pattern.

2.3.7 Distance

Table 7: Delay rate by distance bin

Distance (miles)	Flights	Delay rate
(0,500]	69,026	24.6%
(500,1e+03]	95,383	24.8%
(1e+03,1.5e+03]	65,435	23.2%
(1.5e+03,2e+03]	18,691	23.4%
(2e+03,3e+03]	45,424	21.0%
(3e+03,5e+03]	652	16.6%

Distance is a continuous variable with thousands of distinct values, so it was grouped into six bins with the function `cut()` (each bin includes its upper limit). The delay rate does not follow a linear trend over most of the distances: up to 2,000 miles it remains practically stable, between 23% and 25%, and it only decreases in the longest bins, to 21.0% between 2,000 and 3,000 miles and 16.6% above 3,000 miles.

The longest bin should be interpreted with caution. It has only 652 flights and, departing from New York, it corresponds almost entirely to the Honolulu routes, operated in similar proportions by Hawaiian Airlines (from JFK) and United (from EWR), plus a few United flights to Anchorage. Since Hawaiian Airlines has the lowest delay rate in the study, the lower rate of this bin may partly reflect the carriers that operate these routes, and not only the distance. A possible operational explanation for the lower rates of long flights is that airlines add a time margin to the schedule of long flights (schedule padding), so that a departure delay can be absorbed during the flight. The variable `distance` was used as a numeric predictor, with the caveat that its effect is weak and non-linear; tree-based models are expected to capture the change above 2,000 miles better. Since the distance is practically determined by the origin–destination pair, `distance` and `dest` carry similar information.

2.3.8 Day of the month and day of the week

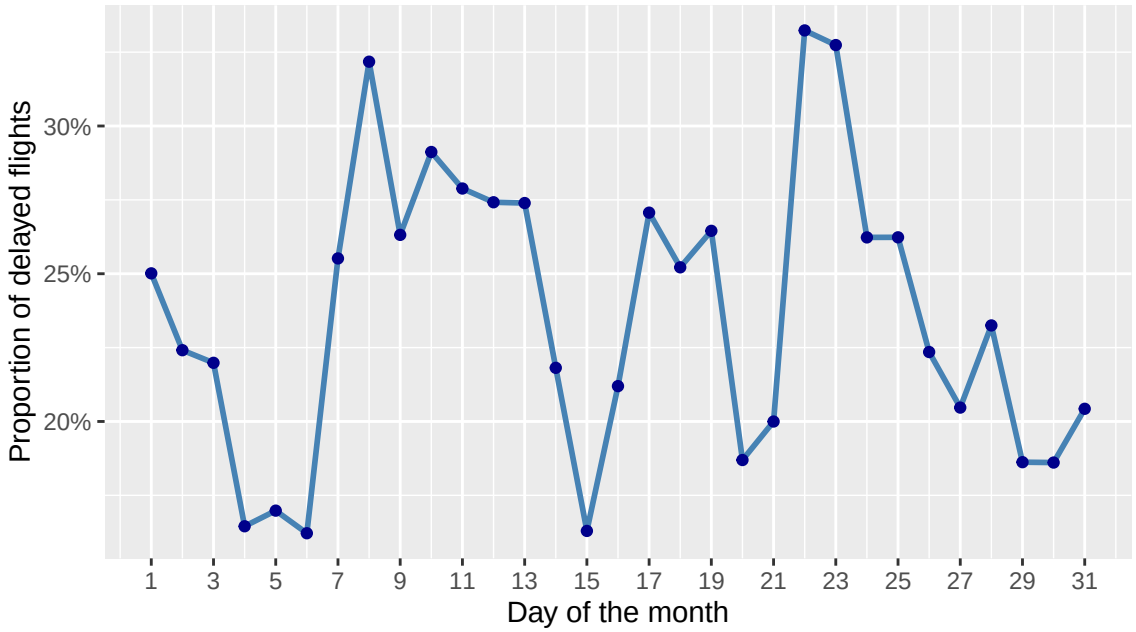


Figure 4: Delay rate by day of the month (development set)

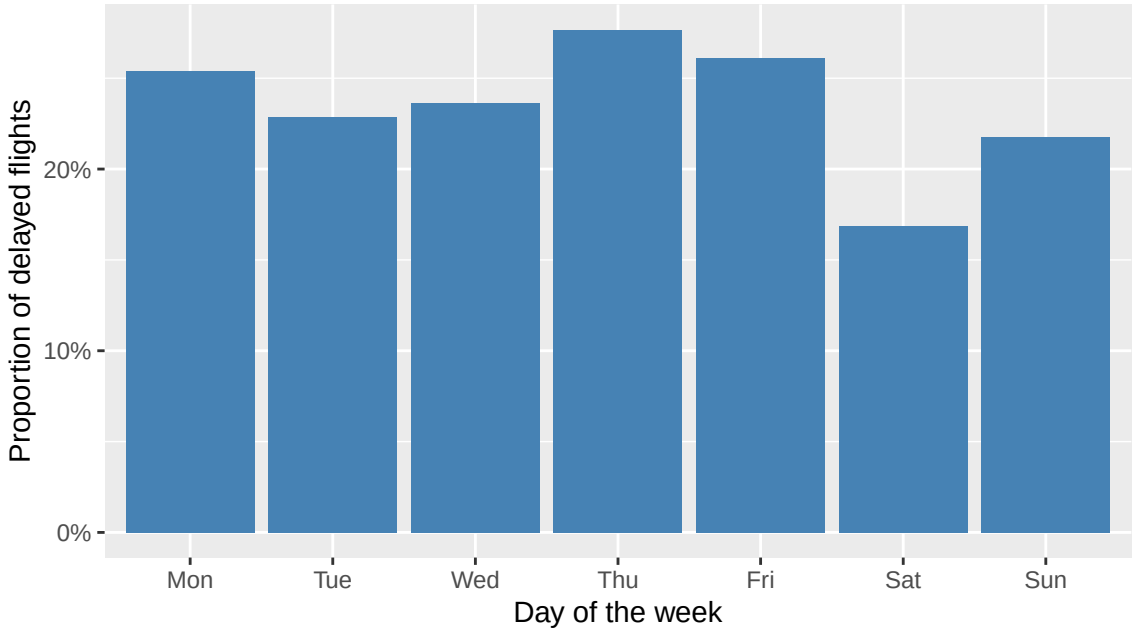


Figure 5: Delay rate by day of the week (development set)

The delay rate by day of the month varies between 16.2% (day 6) and 33.2% (day 22), with isolated peaks (days 8, 22 and 23) and troughs (days 4 to 6 and day 15), but no regular trend or cycle. All days have more than 5,000 flights, with a 95% margin of error of at most about $\pm$ 1.1 percentage points, so these differences are not sampling noise. Each point, however, combines the same calendar day across twelve different months (day 8 combines January 8, February 8 and so on), so a single event on a specific date, such as a strong storm, raises the whole point. The differences are real, but they reflect events of specific dates of 2013, which should not repeat in other years.

There is also a risk of data leakage: together, `day` and `month` identify the exact date of each flight. Since the split between `dev_set` and `final_test` was random within the same year, the test set contains flights from the same dates as the development set, and a tree-based model could learn rules such as "flights on December 22 were very late" and correctly predict test flights affected by the same event. The test performance would look better, but the model would not be useful for future dates. For this reason, `day` was removed from the models and replaced by the day of the week (`weekday`), a pattern that repeats every week. It was created with the functions `make_date()` and `wday()` of the lubridate package, with the week starting on Monday and the day names set in English.

The day of the week shows a clear pattern. Thursday has the highest delay rate (27.7%), followed by Friday (26.1%) and Monday (25.4%), while Tuesday (22.9%) and Wednesday (23.6%) are intermediate. Sunday (21.8%) is the second most punctual day, and Saturday is by far the most punctual one (16.9%). Saturday also has fewer scheduled flights (34,037, against about 44,000 on each weekday), which is consistent with less congestion at the airports and in the airspace. On the weekdays, however, the number of flights is practically the same, while the delay rates differ clearly, so volume alone does not explain the differences. A possible explanation is aircraft load: flights on Thursdays, Fridays and Mondays may carry more passengers and baggage, because of business travel and weekend trips, which would lengthen boarding and increase the chance of cascading delays. The dataset has no occupancy data, so this hypothesis cannot be verified. Since the rate oscillates over the week without a linear trend, `weekday` was used as a factor.

2.3.9 Summary of the exploration

Table 8: Difference between the highest and lowest delay rates of each predictor (percentage points)

Predictor	Range of delay rates
Scheduled hour	26.8 pp
Month	19.4 pp
Destination (at least 1,000 flights)	17.3 pp
Carrier (> 10,000 flights)	13.5 pp
Day of the week	10.8 pp
Distance	8.3 pp
Origin	3.2 pp

The scheduled hour and the month show the largest differences in delay rates, followed by the destination, the carrier and the day of the week, while the distance and, above all, the origin airport show the smallest. This comparison is univariate, since each predictor is analysed in isolation, and it uses different volume criteria for some predictors, so it is only an indication of their relative strength, not a definitive measure of their importance in the models.

2.4 Feature engineering and rare categories

Based on the decisions of the exploration, the same feature engineering was applied to both sets. The day of the month (`day`) was replaced by the day of the week (`weekday`), and the scheduled departure time (`sched_dep_time`) was replaced by the scheduled hour (`sched_hour`), used as a numeric variable; the original columns were then discarded. The variables `month` and `origin` were converted into factors. The final predictors are therefore `month`, `weekday`, `sched_hour`, `carrier`, `origin`, `dest` and `distance`.

Rare categories were also treated. The 13 destinations with fewer than 100 flights in the development set were grouped into a single category, "Other" (392 flights), to avoid unstable estimates for very small groups; as a result, `dest` went from 104 to 92 categories. A check before grouping confirmed that every destination in the final test set also appears in the development set, so the grouping was needed only to deal with instability, and not with unseen destinations. The same principle was evaluated for the carriers, but a threshold of 100 flights would group only SkyWest Airlines (OO), which would not solve anything, while a high threshold such as 1,000 flights would merge carriers with opposite behaviors, such as Hawaiian Airlines (the most punctual) and Frontier Airlines (the most delayed). All carriers were therefore kept, and OO, with only 26 flights, was accepted as a known limitation, with a negligible impact on the overall results.

To avoid data leakage, the rare categories were identified using the development set only. In the final test set, the factors use exactly the same levels as in the development set, and rare destinations are converted to "Other". It was verified that there are no new carriers in the final test set, that the factor levels are identical in both sets and that the final test set has no missing values.

2.5 Validation strategy

The development set was split again, with stratification by the target variable, into a training part (`train_set`, 80%, 235,688 flights) and a validation part (`valid_set`, 20%, 58,923 flights). Because of the computing time required on a conventional computer, the models were trained on a stratified sample of 50,001 flights from the training part (`train_sample`).

These proportions were chosen for practical reasons. The 80/20 split keeps most of the development data for training, while the validation set, with almost 59,000 flights, is large enough to compare the models precisely (the 95% margin of error of an accuracy around 76% is about ± 0.35 percentage points). The sample of about 50,000 flights made it possible to tune all models, including the random forest, which requires training

15 forests in the cross-validation, in several minutes on a conventional computer, while still providing a large number of flights for each model. Five folds were used in the cross-validation as a balance between computing time and stability: each fold leaves out about 10,000 flights to evaluate the model, enough for stable estimates of the AUC and of the balanced accuracy.

Each part has a specific role. A 5-fold cross-validation within the training sample was used to tune the hyperparameters of the models and to choose their classification thresholds. The validation set was used independently to compare the models with each other. The final test set remained isolated and was used only once, at the end, with the chosen model. The proportion of delayed flights stayed at about 23.7% in all subsets, and no category was left without flights in the training sample, although OO kept only 2 flights, which reinforces the limitation described above.

2.6 Models

Four models were compared: a baseline that always predicts "not delayed", a logistic regression, a decision tree (CART) and a random forest. Hyperparameters were chosen by 5-fold cross-validation on the training sample, using the area under the ROC curve (AUC). The AUC was preferred to accuracy because accuracy favors models that predict almost every flight as "not delayed", given the class imbalance, while the AUC measures the ability of a model to rank flights by risk, regardless of the threshold, which is chosen in a separate step.

The algorithms were chosen to increase in complexity, each one addressing a weakness of the previous one. The logistic regression is a simple and interpretable reference model, whose coefficients show the direction of the effect of each predictor, but it assumes additive effects and cannot represent interactions unless they are specified explicitly. The decision tree can capture non-linear patterns, such as the drop in delays above 2,000 miles, and interactions, such as the carrier and airport interaction found in the exploration, but a single tree is unstable. The random forest, which goes beyond linear and logistic regression, reduces this instability by averaging many trees, each trained on a different sample of flights and of variables.

The baseline sets the minimum level of performance. On the validation set, it has a balanced accuracy of 0.500, a sensitivity of 0 and a specificity of 1: it does not identify any delayed flight. Any useful model must therefore reach a balanced accuracy above 0.5 and a sensitivity above 0. Since only information known before departure is available, and factors such as last-minute weather and delays inherited from previous flights are not in the data, the models were expected to improve on the baseline only modestly.

2.6.1 Classification threshold

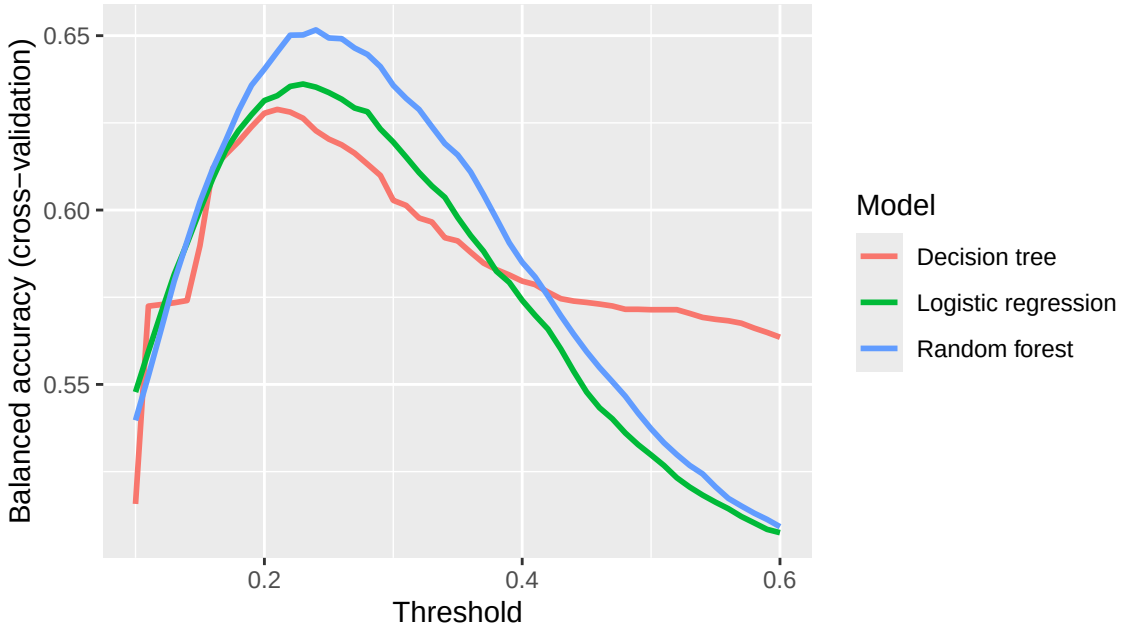


Figure 6: Cross-validated balanced accuracy by classification threshold

The models estimate a probability of delay for each flight, and a flight is classified as delayed when this probability exceeds a threshold. With the default threshold of 0.5, the logistic regression predicted a delay for only 3.6% of the flights, although 23.7% are actually delayed, and identified only 7.9% of the delayed flights. With imbalanced classes, the models rarely estimate a probability above 50%, so this threshold is inadequate.

For each model, thresholds from 0.10 to 0.60, in steps of 0.01, were tested on the out-of-fold predictions of the cross-validation, and the threshold that maximizes the balanced accuracy was chosen. The validation set was not used in this choice. The figure shows the resulting curves. The chosen thresholds were 0.23 for the logistic regression, 0.21 for the decision tree and 0.24 for the random forest, all very close to the proportion of delayed flights (0.237). The decision tree curve is flat at 0.5 for low thresholds and then jumps, because a tree gives the same probability to all flights in the same leaf: below the smallest leaf probability, every flight is classified as delayed.

2.6.2 Logistic regression

The logistic regression was trained with the seven predictors on the training sample, with a cross-validated AUC of 0.685. With the default threshold of 0.5, its accuracy (0.764) was not statistically higher than that of the baseline ($p = 0.21$), and its balanced accuracy was only 0.528. With the threshold of 0.23, chosen by cross-validation, the results on the validation set were: balanced accuracy 0.634, sensitivity 0.660, specificity 0.607 and Kappa 0.203. The balanced accuracy estimated by cross-validation (0.636) is practically the same as on the validation set, which indicates that the threshold generalizes well.

The overall accuracy falls below that of the baseline, because identifying more delays also generates more false alarms. This trade-off is expected and reinforces the choice of balanced accuracy as the main metric. In practice, when the model predicts a delay it is correct in 34.3% of the cases, and when it predicts an on-time flight it is correct in 85.2%, so it is more useful to rule out low-risk flights than to confirm delays.

The coefficient of `sched_hour` (0.109) agrees with the exploration: the odds of delay are multiplied by about 1.11 for each later hour of scheduled departure. The coefficient of `distance` (0.00075), however, is positive, the opposite of the decreasing trend seen in the exploration. Since the distance is practically determined by the origin–destination pair, the effects of `dest` and `origin` absorb most of the information of the distance. This multicollinearity makes the coefficient of `distance` unstable, so it should not be interpreted in isolation. It affects only the interpretation of the coefficients, not the quality of the predictions, and it is another argument for testing tree-based models.

2.6.3 Decision tree

Table 9: Decision tree: cross-validated AUC by complexity parameter (cp)

cp	AUC
0e+00	0.665
2e-05	0.667
5e-05	0.668
1e-04	0.671
2e-04	0.656

The decision tree (CART, `rpart` package) was trained on the training sample, with the complexity parameter (`cp`) chosen by cross-validation. In a first search, values of `cp` of 0.002 and above produced trees with no splits at all (AUC of 0.5), which shows that the patterns in these data are weak and only captured by large trees. Since the best value was repeatedly at the lower limit of the grid, the search was extended down to $\texttt{cp} = 0$, a fully grown tree. As the table shows, the AUC increases up to $\texttt{cp} = 0.0001$ (0.671) and decreases for smaller values (0.665 for $\texttt{cp} = 0$), indicating overfitting in the largest trees. The chosen value is therefore inside the grid.

With the threshold of 0.21, the tree reached a balanced accuracy of 0.629 in cross-validation and 0.622 on the validation set, with a sensitivity of 0.626, a specificity of 0.619 and a Kappa of 0.189. Despite its ability to capture interactions, such as the carrier and airport interaction found in the exploration, the tree performed worse than the logistic regression in AUC and balanced accuracy. Two features of these results help explain this. First, a tree gives the same probability to all flights in the same leaf, so it ranks the flights by risk only in steps; this is visible in the step-shaped curve of the tree in the threshold figure, and it directly lowers the AUC, which measures exactly this ranking. Second, the tree was sensitive to its size: the cross-validated AUC fell from 0.671 to 0.656 when `cp` went from 0.0001 to 0.0002, and to 0.665 with a fully grown tree. A random forest addresses both problems: averaging the probabilities of many trees produces a much finer ranking of the flights and reduces the dependence on the structure of a single tree.

2.6.4 Random forest

Table 10: Random forest: cross-validated AUC by mtry (200 trees, Gini split rule, minimum node size 20)

mtry	AUC
5	0.703
11	0.705
20	0.699

The random forest combines many decision trees, each grown on a bootstrap sample of the flights and considering a random subset of the variables at each split. It was trained with the `ranger` package, because

the classic `randomForest` package does not accept categorical variables with more than 53 levels, and the destination has 92 categories. To keep the computing time manageable, the number of trees was set to 200. The Gini split rule was used, and the minimum number of flights in each terminal node was set to 20, to avoid leaves with very few flights, which would harm the estimated probabilities.

The number of variables sampled at each split (`mtry`) was chosen by cross-validation among 5, 11 and 20, values set around the square root of the number of columns after the categorical variables are converted into indicator columns (about 130). The best value was `mtry` = 11, inside the grid, with an AUC of 0.705. The differences between the values were small (from 0.699 to 0.705), indicating little sensitivity to this parameter in the tested range. With the threshold of 0.24, the random forest reached a balanced accuracy of 0.652 in cross-validation and 0.650 on the validation set, values that are practically the same, which indicates a stable model without overfitting.

3 Results

3.1 Model comparison on the validation set

Table 11: Model comparison on the validation set

Model	AUC (CV)	Threshold	Balanced accuracy	Sensitivity	Specificity	Kappa	PPV	NPV
Baseline	—	—	0.500	0.000	1.000	0.000	—	0.763
Logistic regression	0.685	0.230	0.634	0.660	0.607	0.203	0.343	0.852
Decision tree	0.671	0.210	0.622	0.626	0.619	0.189	0.338	0.842
Random forest	0.705	0.240	0.650	0.646	0.655	0.238	0.368	0.856

The table above was built directly from the objects created by the script (confusion matrices, cross-validated AUCs and chosen thresholds), which guarantees consistency between the code and this report. The positive predictive value (PPV) of the baseline is undefined, because it never predicts a delay, and is shown as "—".

All models outperformed the baseline. The random forest had the best performance, with the highest AUC (0.705), balanced accuracy (0.650), specificity (0.655), Kappa (0.238) and predictive values. The logistic regression had the highest sensitivity (0.660), but with a lower specificity (0.607), that is, more false alarms, a trade-off that justifies the use of balanced accuracy as the main metric. The random forest clearly outperformed the single decision tree (0.650 against 0.622), which is consistent with the idea that combining many trees reduces instability, but its gain over the logistic regression (0.634) was modest. In terms of flights, on the 58,923 flights of the validation set, the random forest raised 2,138 fewer false alarms than the logistic regression (15,525 against 17,663 on-time flights flagged as delayed), while detecting only 202 fewer delayed flights (9,021 against 9,223). The gain of the random forest therefore comes almost entirely from fewer false alarms, which matters for an airline that would allocate resources to the flagged flights. These limited gains, with a balanced accuracy of about 0.65 for the best model, reflect the difficulty of predicting delays using only information known before departure. The random forest was therefore chosen as the final model.

3.2 Variable importance

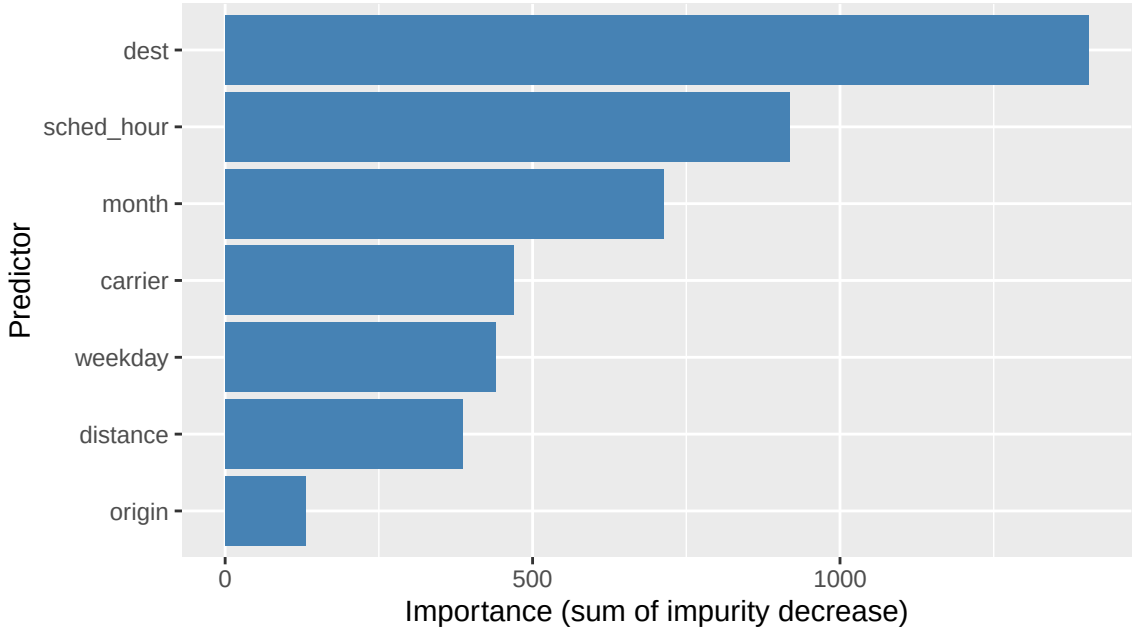


Figure 7: Random forest variable importance, summed by original predictor

The `ranger` package computes the impurity importance (decrease in Gini impurity) for each indicator column created from the categorical variables, so the importances were summed back into the seven original predictors. The resulting ranking is: dest (1,404), sched_hour (918), month (713), carrier (470), weekday (439), distance (387), origin (132).

From the fourth to the seventh position, this ranking matches exactly the order of the univariate ranges of delay rates observed in the exploration. The origin airport is in last place, confirming its low strength, and the distance is in the second to last place, which is consistent with its redundancy with the origin–destination pair. The first position of `dest` should be interpreted with caution: impurity importance tends to favor numeric variables, such as `sched_hour`, and categorical variables with many categories, such as `dest`, because both offer many possible split points. The ranking therefore shows that the model used the destination intensively to split the flights, but not, by itself, that the destination is the main cause of delays.

3.3 Final model and evaluation on the final test set

Table 12: Final random forest on the final test set

Metric	Value
Accuracy	0.660
Sensitivity	0.638
Specificity	0.667
Balanced accuracy	0.652
Kappa	0.245
Positive predictive value	0.373
Negative predictive value	0.855

The random forest was retrained a single time on the complete development set (294,611 flights), with the hyperparameters chosen by cross-validation kept fixed (`mtry` = 11, Gini split rule, minimum node size of 20 and 200 trees) and without any new tuning. The same principle was used in the MovieLens project: tuning the model on part of the data and retraining it on all the development data before the final evaluation. The threshold of 0.24, chosen by cross-validation, was also kept. The final test set (32,735 flights) remained isolated during the whole development and was used only once, with no adjustment after seeing the results.

On the final test set, the model reached a balanced accuracy of 0.652, practically the same as on the validation set (0.650), which indicates that it generalizes well to unseen flights from the same period. The small improvement may be related to the retraining with more data, but it is within the margin of error: the 95% confidence interval of the accuracy (from 0.655 to 0.665) corresponds to a margin of about $\pm$ 0.5 percentage points, consistent with the precision expected for this test set size. Since the final test set was drawn from the same year as the training data, these results do not show the performance of the model in future years.

In practical terms, the final model identifies about 64% of the flights that will arrive more than 15 minutes late, using only information known before departure from New York. When it predicts a delay, it is correct in 37% of the cases; when it predicts an on-time flight, it is correct in 86% of the cases.

4 Conclusion

This project developed a model to predict the arrival delays of commercial flights departing from the three airports of the New York City metropolitan area in 2013. After cleaning the data, exploring the predictors, engineering new features and choosing the decision thresholds by cross-validation, four approaches were evaluated. The random forest, implemented with the `ranger` package, had the best performance, outperforming the baseline, the logistic regression and the single decision tree. Evaluated a single time on the independent final test set, the retrained random forest reached an accuracy of 0.660, a sensitivity of 0.638, a specificity of 0.667, a balanced accuracy of 0.652 and a Kappa of 0.245. These results show that, even using only static information known before departure, the model captured consistent patterns found in the exploration: delays become more frequent as the scheduled hour advances, they are concentrated in December and in the summer months, they are more common on Thursdays, Fridays and Mondays than on Saturdays, and they vary with the destination and the carrier.

Potential impact. In practice, the random forest could be used by airlines as a risk score computed in advance for each scheduled flight, since all its predictors are known well before departure (for example, 24 to 48 hours ahead). Its main practical value lies in identifying low-risk flights: when the model predicts that a flight will be on time, it is correct in 85.5% of the cases on the final test set. Operations managers could therefore focus contingency resources, such as spare crews and ground buffers, on the flights flagged as high risk. Even with a modest positive predictive value of 37.3%, which reflects the limits of predicting dynamic events with static pre-departure data, the flights flagged by the model are delayed about 1.6 times as often as the average flight (23.7%). This could support crew and ground planning and help reduce the risk that a local disruption spreads through an airline's network.

Limitations. The model has important limitations that restrict its performance and its generalization. First, it uses only information known before departure, leaving out dynamic factors such as real-time weather (snowstorms or thunderstorms), sudden mechanical problems and the cascade of delays carried by the rotation of the aircraft. Second, the data cover only 2013, and the final test set was drawn from the same year, so the patterns learned by the model, such as the seasonal ones, may be specific to that year, and its performance in future years was not evaluated. Third, because of the computing cost, the choice of hyperparameters and the comparison of the models used a sample of 50,001 flights; a search with more data could lead to better hyperparameters. Fourth, carriers with very few flights, such as SkyWest Airlines (OO), with only 2 flights in the training sample, produce unstable estimates. Finally, cancelled flights were removed during cleaning and were not modelled, and the impurity importance used to rank the predictors has a known bias in favor of variables with many categories (such as the destination) and numeric variables (such as the scheduled hour).

Future work. Three main directions are proposed to go beyond the performance limit of the current approach:

- **Weather data:** adding historical weather observations for the airports, such as METAR reports with visibility, wind and thunderstorm activity. Weather forecasts available before departure would keep the model usable in advance, while observations closer to the departure time could improve predictions over shorter horizons.
- **Aircraft rotation and delay propagation:** the aircraft tail number (`tailnum`) could be used to reconstruct the sequence of flights of each aircraft. Knowing whether the incoming flight of the same aircraft is late would capture delay propagation, one of the main operational causes of late departures. This would require data on the flights arriving in New York, which are not part of this dataset.
- **Validation over time:** instead of a random split within the same year, training the model on 2013 data and testing it on later years would show how well it adapts to long-term changes in airline networks and airport capacity.

5 References

- Ball, M., Barnhart, C., Dresner, M., Hansen, M., Neels, K., Odoni, A., Peterson, E., Sherry, L., Trani, A. and Zou, B. (2010). *Total Delay Impact Study: A Comprehensive Assessment of the Costs and Impacts of Flight Delay in the United States.* NEXTOR, report for the Federal Aviation Administration. https://rosap.ntl.bts.gov/view/dot/6234
- U.S. Bureau of Transportation Statistics. *Airline On-Time Performance Data* (original source of the flight records).
- Wickham, H. *nycflights13: Flights that Departed NYC in 2013.* R package (data from the U.S. Bureau of Transportation Statistics). https://CRAN.R-project.org/package=nycflights13
- mahoora00135. *Flights* [dataset]. Kaggle. License: CC0 Public Domain. https://www.kaggle.com/datasets/mahoora00135/flights
- Irizarry, R. A. *Introduction to Data Science: Data Analysis and Prediction Algorithms with R.* In particular, Sections 18.2 and 19.4 (confounding), 27.4 (evaluation metrics, including balanced accuracy), 29 (cross validation), 30 (the caret package), 31.3 (logistic regression), 31.10 and 31.11 (decision trees and random forests) and 32.3 (variable importance).
- Kuhn, M. *caret: Classification and Regression Training.* R package. https://CRAN.R-project.org/package=caret
- Wright, M. N. and Ziegler, A. (2017). ranger: A Fast Implementation of Random Forests for High Dimensional Data in C++ and R. *Journal of Statistical Software*, 77(1), 1–17.
- Therneau, T. and Atkinson, B. *rpart: Recursive Partitioning and Regression Trees.* R package. https://CRAN.R-project.org/package=rpart
- R Core Team. *R: A Language and Environment for Statistical Computing.* R Foundation for Statistical Computing, Vienna, Austria.
- Anthropic. Claude (generative AI tool), used during this project as described in the note on sources at the top of `aviation.R`.